

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: cap = cv2.VideoCapture(0)

if cap.isOpened():
    print("Webcam Connected Successfully")
else:
    print("Webcam Not Found")

cap.release()
```

Webcam Connected Successfully

```
In [3]: import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()

old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)

feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

lk_params = dict(
    winSize=(15,15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

mask = np.zeros_like(old_frame)

while True:

    ret, frame = cap.read()

    if not ret:
```

```

        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    p1, st, err = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is not None:

        good_new = p1[st==1]
        good_old = p0[st==1]

        for i, (new, old) in enumerate(
            zip(good_new, good_old)
        ):

            a, b = new.ravel()
            c, d = old.ravel()

            mask = cv2.line(
                mask,
                (int(a), int(b)),
                (int(c), int(d)),
                (0, 255, 0),
                2
            )

            frame = cv2.circle(
                frame,
                (int(a), int(b)),
                5,
                (0, 0, 255),
                -1
            )

            img = cv2.add(frame, mask)

            cv2.imshow(
                "Lucas Kanade Optical Flow",
                img
            )

            old_gray = frame_gray.copy()

            p0 = good_new.reshape(
                -1,
                1,
                2
            )

    if cv2.waitKey(30) & 0xFF == 27:
        break

```

```
cap.release()  
cv2.destroyAllWindows()
```

```
In [4]: import cv2  
import numpy as np  
  
cap = cv2.VideoCapture(0)  
  
ret, frame1 = cap.read()  
  
prvs = cv2.cvtColor(  
    frame1,  
    cv2.COLOR_BGR2GRAY  
)  
  
hsv = np.zeros_like(frame1)  
  
hsv[...,1] = 255  
  
while True:  
  
    ret, frame2 = cap.read()  
  
    if not ret:  
        break  
  
    next_frame = cv2.cvtColor(  
        frame2,  
        cv2.COLOR_BGR2GRAY  
    )  
  
    flow = cv2.calcOpticalFlowFarneback(  
        prvs,  
        next_frame,  
        None,  
        0.5,  
        3,  
        15,  
        3,  
        5,  
        1.2,  
        0  
    )  
  
    mag, ang = cv2.cartToPolar(  
        flow[...,0],  
        flow[...,1]  
    )  
  
    hsv[...,0] = ang * 180 / np.pi / 2  
  
    hsv[...,2] = cv2.normalize(  
        mag,  
        None,  
        0,  
        255,  
        cv2.NORM_MINMAX  
    )
```

```

    rgb = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    cv2.imshow(
        "Farneback Dense Optical Flow",
        rgb
    )

    prvs = next_frame

    if cv2.waitKey(30) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()

```

```

In [5]: comparison = {
    "Lucas-Kanade":
    {
        "Type": "Sparse",
        "Speed": "Fast",
        "Complexity": "Low",
        "Tracking": "Excellent"
    },

    "Farneback":
    {
        "Type": "Dense",
        "Speed": "Moderate",
        "Complexity": "Higher",
        "Tracking": "Pixel-wise Motion"
    }
}

for k,v in comparison.items():

    print("\n",k)

    for key,val in v.items():

        print(
            f"{key}: {val}"
        )

```

```

Lucas-Kanade
Type: Sparse
Speed: Fast
Complexity: Low
Tracking: Excellent

```

```

Farneback
Type: Dense
Speed: Moderate
Complexity: Higher
Tracking: Pixel-wise Motion

```

```

In [6]: print("OBSERVATIONS")
        print("-"*50)

```

```
print("1. Lucas-Kanade tracks selected feature points.")
print("2. Farneback estimates motion for every pixel.")
print("3. Lucas-Kanade is computationally efficient.")
print("4. Farneback provides richer motion information.")
print("5. Camera motion affects optical flow accuracy.")
print("6. Lighting variations can introduce noise.")
```

OBSERVATIONS

-
1. Lucas-Kanade tracks selected feature points.
 2. Farneback estimates motion for every pixel.
 3. Lucas-Kanade is computationally efficient.
 4. Farneback provides richer motion information.
 5. Camera motion affects optical flow accuracy.
 6. Lighting variations can introduce noise.